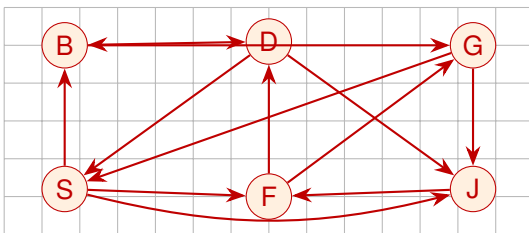


3.2 Graphen implementieren – die Adjazenzmatrix

Beim Ausflug in einen Vergnügungspark werden die Sitzplätze der Achterbahn verteilt. Björn, Doris, Fatma, Gerda, Jakob und Simon haben mit einem Kreuz notiert, mit wem sie gerne fahren möchten. Gelesen wird zeilenweise: Björn (Zeile B) möchte mit Doris und mit Gerda fahren.

	B	D	F	G	J	S
B		X		X		
D	X				X	X
F		X		X		
G					X	X
J			X			
S	X		X		X	

Aufgabe 1) Achterbahn



a) Begründe, dass die Tabelle einen Graphen darstellt, und zeichne ihn links.

Personen = Knoten, Kreuz = Kante von Zeile zu Spalte

b) Ist der Graph gerichtet? Begründe mit einem Beispiel.
ja: B möchte mit G fahren, G aber nicht mit B.

c) Wer sollte auf jeden Fall nebeneinandersitzen?
nur Björn und Doris: Sie haben sich gegenseitig gewählt.

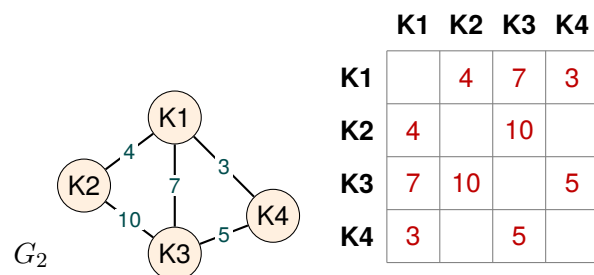
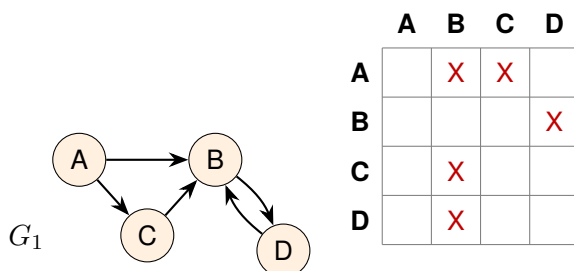
Die Adjazenzmatrix

Ein Graph wird im Computer meist als **Adjazenzmatrix** (adjacent = benachbart) gespeichert, einer Tabelle wie oben. Zeilen und Spalten stehen für die Knoten.

- Führt eine Kante von Knoten i nach Knoten j , steht in Zeile i und Spalte j ein X (in Java: true).
- Bei gewichteten Graphen steht dort statt des X das Gewicht der Kante.
- Bei ungerichteten Graphen ist die Matrix symmetrisch zur Hauptdiagonalen: Jede Kante $\{i, j\}$ steht zweimal in der Matrix, bei (i, j) und bei (j, i) .

Aufgabe 2)

a) Gib zu G_1 und G_2 die Adjazenzmatrix an. b) Vergleiche die Matrizen. Was fällt bei G_2 auf?



b) G_1 : X-Einträge; G_2 : Gewichte, Matrix symmetrisch zur Hauptdiagonalen, da ungerichtet.

Implementierung in Java Website: Projekt Graph

```
public class Graph {
    Knoten[] knoten; // Knotenfeld
    boolean[][] adja; // Adjazenzmatrix
    int anzKnoten = 5;

    Graph() {
        knoten = new Knoten[anzKnoten];
        adja = new boolean[anzKnoten][anzKnoten];
        // ... Knoten erzeugen
    }
}
```

Die Matrix ist ein Feld von Feldern (jede Zeile ein Feld): ein zweidimensionales Array

Die Knoten werden über ihre Nummer (Index) angesprochen. Die Knoten-Objekte liegen im Knotenfeld an derselben Position. adja muss nicht befüllt werden, boolean ist anfangs false.

Aufgabe 3) Starte das Projekt auf der Website unverändert. Was meldet die IDE, und warum?

Fehler: knoten ist null

Ergänze dann TODO 1 wie im Code links.

Aufgabe 4) Projekt Graph

Übertrage die Methode ins Projekt (TODO 2). Sage vorher, was bei den Aufrufen passiert, und teste.

```
void kanteHinzufuegen(int start, int ende) {
    if (start > anzKnoten || ende > anzKnoten) {
        println("Einfügen nicht möglich.");
    } else {
        adja[start][ende] = true;
        zeichneKante(start, ende);
    }
    printAdja();
}
```

Aufruf	Was passiert?
(0, 1)	adja[0][1] = true, Kante 0 → 1
(4, 2)	adja[4][2] = true, Kante 4 → 2
(7, 1)	7 > 5: „Einfügen nicht möglich.“
(5, 0)	5 > 5 falsch: adja[5][0] ⇒ Absturz, Index 5 gibt es nicht

b) Für welche Art von Graphen passt die Methode? für gerichtete Graphen

c) Verbessere sie: keine ungültigen Nummern, ungerichtete Kanten. Notiere nur die geänderten Zeilen.

```
if (start < 0 || start >= anzKnoten || ende < 0 || ende >= anzKnoten) {
    adja[start][ende] = true; adja[ende][start] = true;
}
```

Aufgabe 5 und 6) Projekt Graph

5) Ergänze printAdja() (TODO 3): Die Matrix soll zeilenweise mit X und O erscheinen.

6) nachbarnAusgeben(int k) gibt alle Knoten aus, zu denen von k eine Kante führt (TODO 4).

```
void printAdja() {
    for (int zeile = 0; zeile < anzKnoten; zeile++) {
        for (int spalte = 0; spalte < anzKnoten; spalte++) {
            if (adja[zeile][spalte]) {
                print("X ");
            } else {
                print("0 ");
            }
        }
        println(); // Zeilenumbruch
    }
    println();
}
```

```
void nachbarnAusgeben(int k) {
    print("Nachbarn von " + k + ": ");
    for (int i = 0; i < anzKnoten; i++) {
        if (adja[k][i]) {
            print(i + " ");
        }
    }
    println();
}
```

Merke: Die Nachbarn von Knoten *k* stehen in Zeile *k*. Man durchläuft sie mit einer Wiederholung mit fester Anzahl und prüft jeden Eintrag.